Algoritmi e Strutture Dati

Capitolo 3
Strutture dati elementari

Camil Demetrescu, Irene Finocchi,
Giuseppe F. Italiano



Gestione di collezioni di oggetti

Tipo di dato (Abstract Data Type): cosa?

- Specifica delle operazioni di interesse su una collezione di oggetti (es. inserisci, cancella, cerca)
- **In Java**: un'interfaccia

Struttura dati: come?

- Organizzazione dei dati che permette di supportare le operazioni di un tipo di dato usando meno risorse di calcolo possibile
- **In Java**: una classe che implementa l'interfaccia associata al tipo di dato

Copyright © 2008 - The McGraw - Hill Companies, srl



Il tipo di dato Dizionario

tipo `Dizionario`:
dati:
un insieme S di coppie $(elem, chiave)$.

operazioni:

`insert`$(elem\ e, chiave\ k)$
aggiunge a S una nuova coppia (e, k).

`delete`$(chiave\ k)$
cancella da S la coppia con chiave k.

`search`$(chiave\ k) \rightarrow elem$
se la chiave k è presente in S restituisce l'elemento e ad essa associato, e `null` altrimenti.

Copyright © 2008 - The McGraw - Hill Companies, srl



Il tipo di dato Dizionario – in Java

PRIMO modo: Object + Comparable

```
public interface Dizionario {
 public void insert(Object e,Comparable k);
 public void delete(Comparable k);
 public Object search(Comparable k);
};
```



Il tipo di dato Dizionario – in Java

SECONDO modo: tipo generico + Comparable<T>

```
public interface Dizionario
  <E, K extends Comparable <? Super K >> {
   public void insert(E e, K k);
   public void delete(K k); // Null if
   none
   public E search(K k);   // Null if none
};
```



Tecniche di rappresentazione dei dati

Rappresentazioni indicizzate:

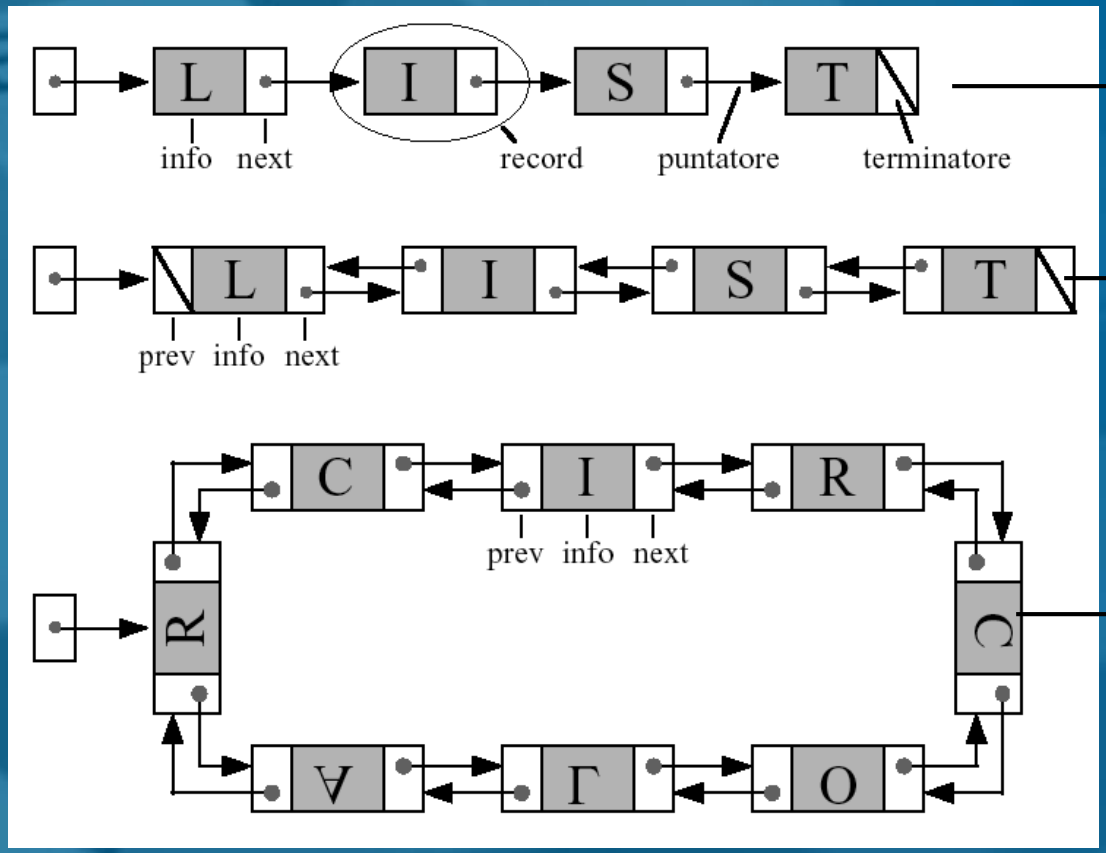
- I dati sono contenuti in array

Rappresentazioni collegate:

- I dati sono contenuti in record collegati fra loro mediante puntatori

Copyright © 2008 - The McGraw - Hill Companies, srl

Esempi di strutture collegate



Lista semplice

Lista doppiamente collegata

Lista circolare doppiamente collegata

Copyright © 2008 - The McGraw - Hill Companies, srl



Pro e contro

Rappresentazioni indicizzate:

- **Pro**: accesso diretto ai dati mediante indici
- **Contro**: dimensione fissa (riallocazione array richiede tempo lineare)

Rappresentazioni collegate:

- **Pro**: dimensione variabile (aggiunta e rimozione record in tempo costante)
- **Contro**: accesso sequenziale ai dati

Copyright © 2008 - The McGraw - Hill Companies, srl

Dizionario realizzato mediante array ordinato

classe `ArrayOrdinato` **implementa** `Dizionario`:

dati: $S(n) = \Theta(n)$

un array S di dimensione n contenente coppie $(elem, chiave)$.

operazioni:

`insert`$(elem\ e, chiave\ k)$ $T(n) = O(n)$

rialloca l'array S aumentandone la dimensione n di uno; cerca il più piccolo indice i tale che $k \leq S[i].chiave$ e pone $S[j] \leftarrow S[j-1]$ per ogni j da $n-1$ a $i+1$; infine, pone $S[i] \leftarrow (e, k)$.

`delete`$(chiave\ k)$ $T(n) = O(n)$

trova l'indice i della coppia con chiave k in S e pone $S[j] \leftarrow S[j+1]$ per ogni j da i a $n-2$; infine, rialloca l'array S diminuendone la dimensione n di uno.

`search`$(chiave\ k) \rightarrow elem$ $T(n) = O(\log n)$

esegue l'algoritmo di ricerca binaria su S per verificare se S contiene la chiave k. Se la ricerca ha successo restituisce l'elemento e associato alla chiave, altrimenti restituisce `null`.

Copyright © 2008 - The McGraw - Hill Companies, srl

Dizionario mediante lista circolare

classe `StrutturaCollegata` **implementa** `Dizionario`:

$S(n) = \Theta(n)$

dati:

una collezione di n record contenenti ciascuno una quadrupla $(elem, chiave, next, prev)$, dove $next$ e $prev$ sono puntatori al successivo e precedente record nella collezione, rispettivamente. Manteniamo inoltre un puntatore $list$ che contiene l'indirizzo di un record se la collezione non è vuota, e `null` altrimenti.

operazioni:

`insert`$(elem\ e, chiave\ k)$ $T(n) = O(1)$

viene creato un record p con elemento e e chiave k. Se $list$ = `null`, si effettua $p.next \leftarrow p$, $p.prev \leftarrow p$ e $list \leftarrow p$. Altrimenti, si collega il record p tra $list$ e $list.next$ effettuando $p.next \leftarrow list.next$, $list.next.prev \leftarrow p$, $p.prev \leftarrow list$ e $list.next \leftarrow p$.

`delete`$(chiave\ k)$ $T(n) = O(n)$

si trova il record p con chiave k come nella `search`; poi, si effettua $p.prev.next \leftarrow p.next$ e $p.next.prev \leftarrow p.prev$; infine, viene distrutto il record p.

`search`$(chiave\ k) \rightarrow elem$ $T(n) = O(n)$

se $list$ = `null` si restituisce `null`. Altrimenti, si scandisce la struttura saltando di record in record con $p \leftarrow p.next$ fino a quando non diventa $p = list$, verificando se qualche p ha chiave k. In caso positivo si restituisce l'elemento trovato, e `null` altrimenti.

Copyright © 2008 - The McGraw - Hill Companies, srl



Il tipo di dato Pila

LIFO: Last In, First Out.

tipo `Pila`:
dati:
una sequenza S di n elementi.

operazioni:

`isEmpty`$() \rightarrow result$
restituisce `true` se S è vuota, e `false` altrimenti.

`push`$(elem\ e)$
aggiunge e come ultimo elemento di S.

`pop`$() \rightarrow elem$
toglie da S l'ultimo elemento e lo restituisce.

`top`$() \rightarrow elem$
restituisce l'ultimo elemento di S (senza toglierlo da S).

Copyright © 2008 - The McGraw - Hill Companies, srl



Implementazione di una Pila mediante array

```
private int maxSize;//Max dimensione
private int top; //Indice elemento top
private Object [] listArray;
```

Questioni:

- Quale estremo dell'array è il top?
  - L'ultimo elemento
- Quali sono i costi delle operazioni?
  - O(1)



Implementazione di una Pila mediante linked list

```
class LPila implements Pila {
   private Record top;//primo elemento della lista
   private int size;
   …
}
```

Questioni:

- Quali sono i costi delle operazioni?
  - O(1) inserendo e cancellando in testa
- Quanto spazio richiede rispetto all'implementazione array-based?
  - Numero effettivo di elementi



Il tipo di dato Coda

FIFO: First in, First Out

tipo `Coda`:
dati:
una sequenza S di n elementi.

operazioni:

`isEmpty`$() \rightarrow result$
restituisce `true` se S è vuota, e `false` altrimenti.

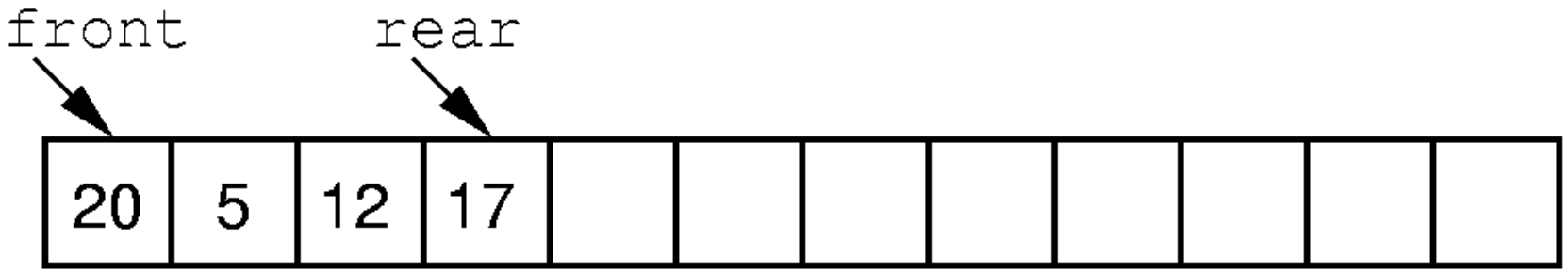
`enqueue`$(elem\ e)$
aggiunge e come ultimo elemento di S.

`dequeue`$() \rightarrow elem$
toglie da S il primo elemento e lo restituisce.

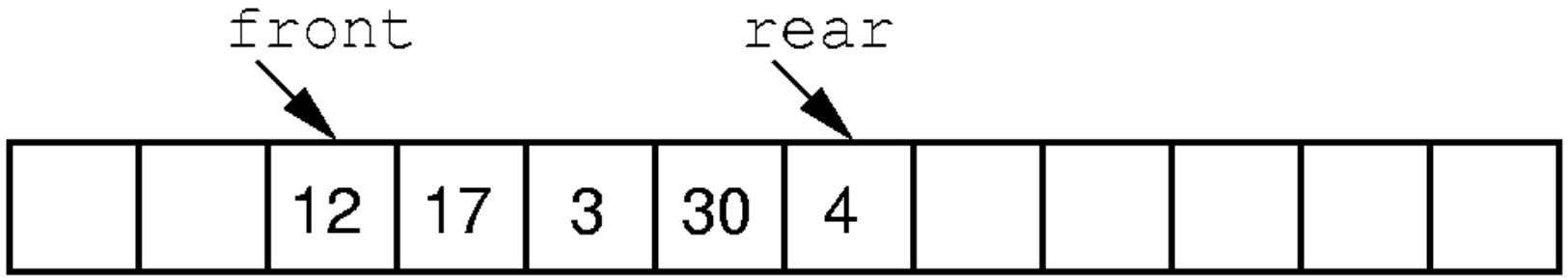
`first`$() \rightarrow elem$
restituisce il primo elemento di S (senza toglierlo da S).

Copyright © 2008 - The McGraw - Hill Companies, srl

Implementazione di una coda mediante array(1)



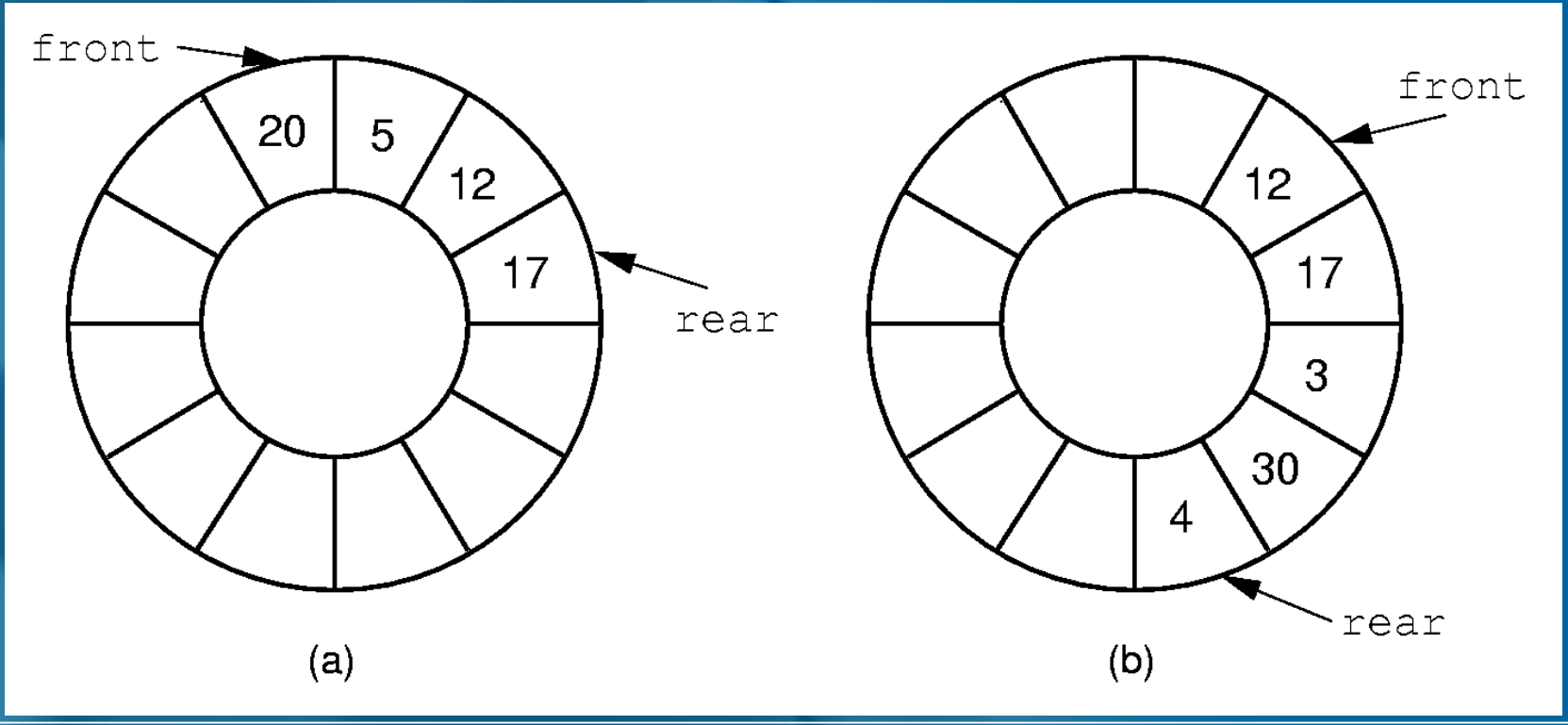
(a)



(b)

Implementazione di una coda mediante array circolare (2)

- Dove puntano front e rear?
- Come distinguiamo tra una coda piena ed una coda vuota?



Implementazione di una coda mediante array circolare (3)

Front, rear: le posizioni occupate dalla coda vanno da front a rear, front precede rear in senso orario

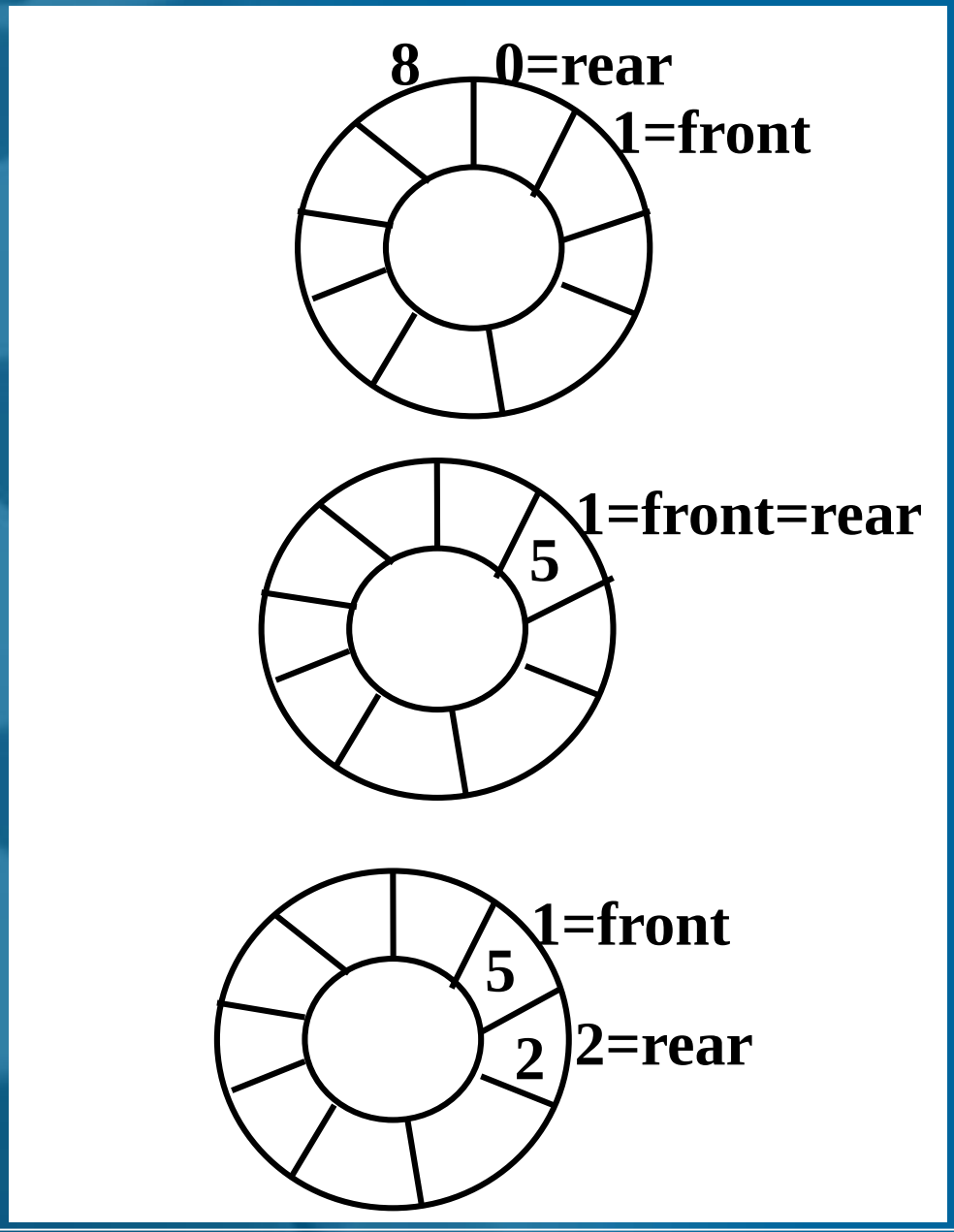


Front=2, rear=5 ➔ coda=(A[2],A[3],A[4],A[5])

Front=5, rear=2 ➔ coda=(A[5],…,A[8],A[0],A[1],A[2])



Coda circolare vuota di dimensione n=9:

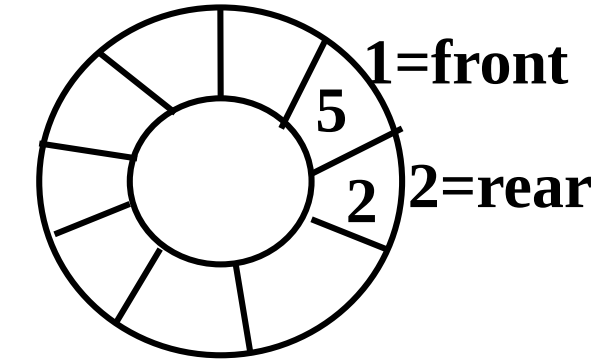


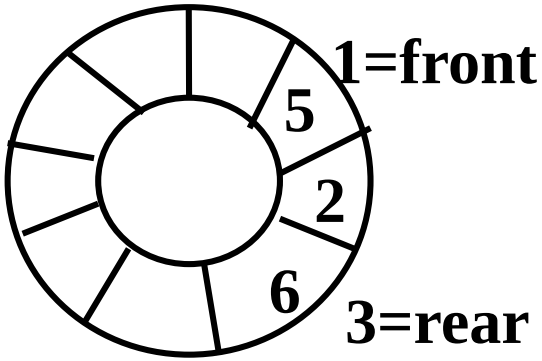
Inserire 5:

Inserire 2:

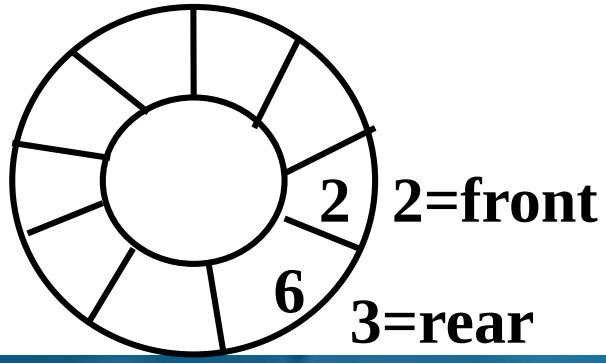


Inserire 6:

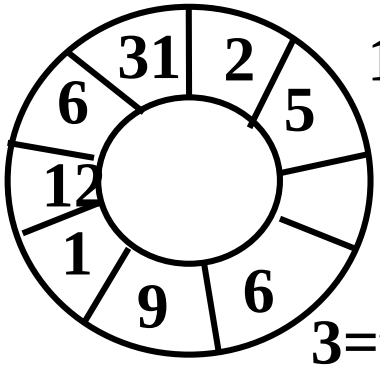




Cancellare:



Coda piena:



Implementazione di una coda mediante array circolare (4)

Coda vuota: front in posizione immediatamente successiva al rear
es. front=3, rear=2.

Coda piena: contiene **n-1 elementi** (e non n)
front a 2 posizioni successive al rear
es. front=3, rear=1 ➔ coda=(A[3],…,A[n-1],A[0],A[1])

Nota: se coda contenesse n elementi allora front in posizione immediatamente successiva al rear,
es. front=3, rear=2 ➔
coda=(A[3],…,A[n-1], A[0], A[1], A[2]), si confonderebbe con coda vuota



Implementazione di una coda mediante array circolare (5)

Clear: poni front=1, rear=0 (quindi front=rear+1 % n)

isEmpty: controlla se front== (rear+1) % n

isFull: risultato del confronto (front== (rear +2) % n)

Enqueue(Q,x): se coda è piena restituisce FALSE, altrimenti rear=(rear +1)%n; Q[rear]=x; return TRUE

Dequeue(Q,x): se coda vuota restituisce null, altrimenti x=Q[front]; front=(front+1)%n; return x



Implementazione di una Coda mediante Linked list

Esempio Coda

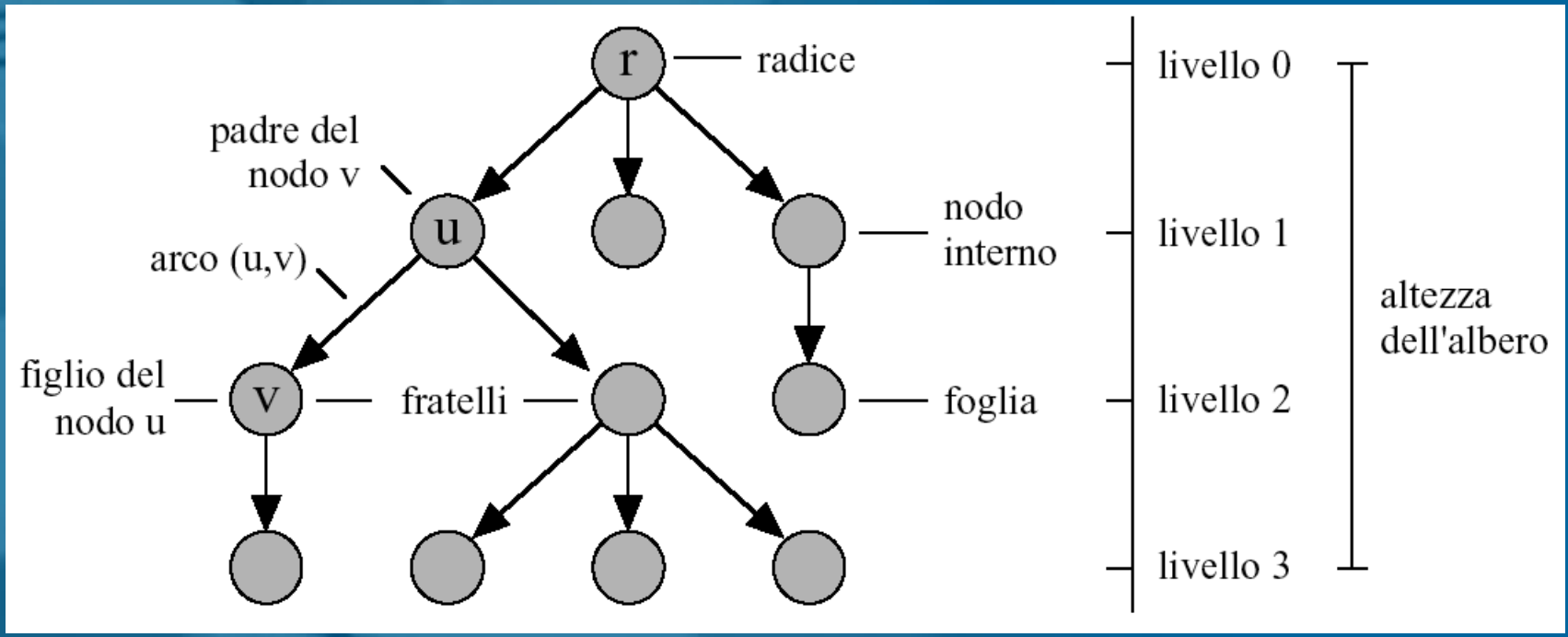
Vedi listati .java:

- Coda
- CodaCollegata
- EccezioneStrutturaVuota
- ProvaCoda (driver test)

Costi operazioni: O(1)
Spazio occupato: numero effettivo di elementi

Alberi

Un albero (radicato) è una **coppia T = (N, A)** costituita da un insieme N di nodi e da un insieme A di coppie di nodi, dette archi.



Organizzazione gerarchica dei dati

Dati contenuti nei nodi, relazioni gerarchiche definite dagli archi che li collegano

Copyright © 2008 - The McGraw - Hill Companies, srl



Alberi: alcune definizioni

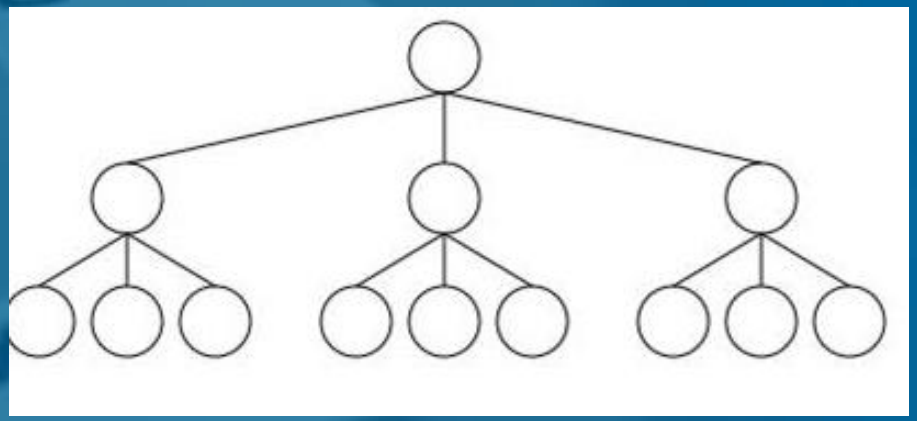
- **Grado di un nodo**: numero di figli del nodo
- **Cammino**: sequenza di nodi $<n_0, n_1, \ldots, n_k>$ dove il nodo n_i è padre del nodo n_{i+1}, per $0 <= i < k$
  - La lunghezza del cammino è k (numero di archi)
  - Dato un nodo, esiste **un unico cammino** dalla radice dell'albero al nodo
- **Livello o profondità di un nodo**: lunghezza del cammino dalla radice al nodo
  - Definizione ricorsiva: il livello della radice è 0, il livello di un nodo non radice è 1 + il livello del padre
- **Altezza dell'albero**: la lunghezza del più lungo cammino nell'albero (la max profondità a cui si trova una foglia)
  - Parte dalla radice e termina in una foglia

Copyright © 2008 - The McGraw - Hill Companies, srl

Alberi k-ari completi

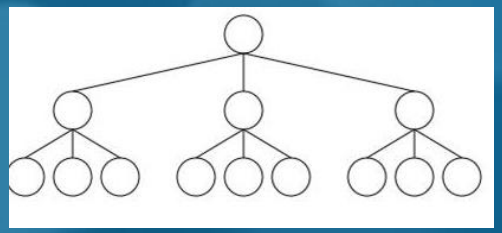
- Un **albero k-ario completo** è un albero k-ario in cui tutte le foglie hanno la stessa profondità e tutti i nodi interni hanno grado k

albero 3-ario completo



Alberi k-ari completi

- Il numero di foglie di un albero k-ario completo:
  - la radice ha k figli a profondità 1
  - ognuno dei figli ha k figli a profondità 2 per un totale di k*k foglie
  - a profondità h (altezza dell'albero) si hanno k^h foglie
- Il numero di nodi interni di un albero k-ario completo di altezza h: $1+k+ k^2 + \ldots +k^{h-1} = \Sigma_{i=0..h-1} k^i = (k^h -1)/(k-1)$



- Un albero binario completo ha $2^h -1$ nodi interni e 2^h foglie



tipo `Albero`:

dati:

un insieme di nodi (di tipo *nodo*) e un insieme di archi.

operazioni:

`numNodi`() $\rightarrow$ *intero*
restituisce il numero di nodi presenti nell'albero.

`grado`(*nodo* v) $\rightarrow$ *intero*
restituisce il numero di figli del nodo v.

`padre`(*nodo* v) $\rightarrow$ *nodo*
restituisce il padre del nodo v nell'albero, o `null` se v è la radice.

`figli`(*nodo* v) $\rightarrow \langle$*nodo*, *nodo*, ..., *nodo*$\rangle$
restituisce, uno dopo l'altro, i figli del nodo v.

`aggiungiNodo`(*nodo* u) $\rightarrow$ *nodo*
inserisce un nuovo nodo v come figlio di u nell'albero e lo restituisce. Se v è il primo nodo ad essere inserito nell'albero, esso diventa la radice (e u viene ignorato).

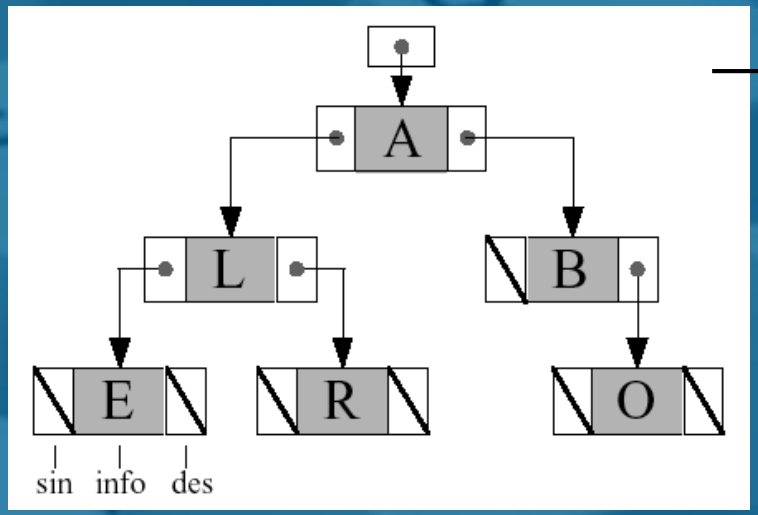
`aggiungiSottoalbero`(`Albero` a, *nodo* u)
inserisce nell'albero il sottoalbero a in modo che la radice di a diventi figlia di u.

`rimuoviSottoalbero`(*nodo* v) $\rightarrow$ `Albero`
stacca e restituisce l'intero sottoalbero radicato in v. L'operazione cancella dall'albero il nodo v e tutti i suoi discendenti.

Copyright © 2008 - The McGraw - Hill Companies, srl

Rappresentazioni collegate di alberi

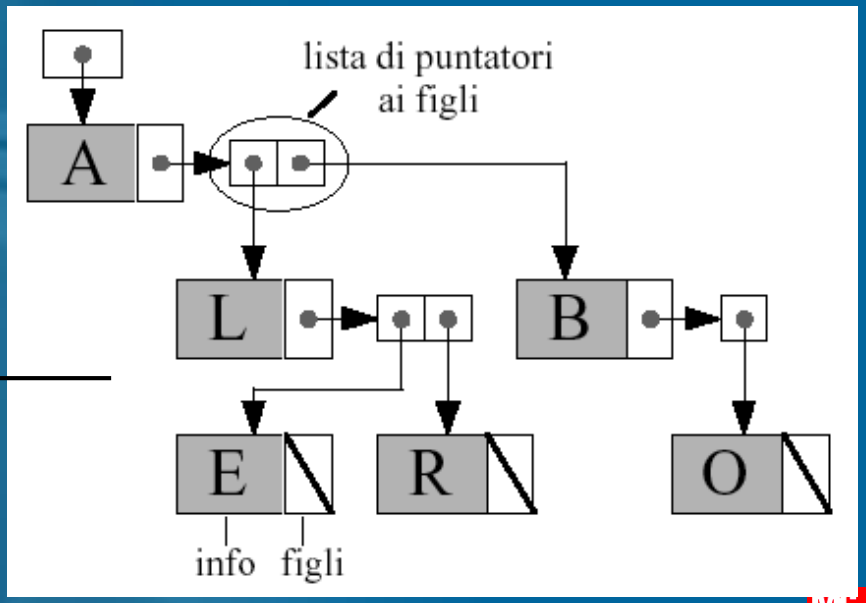
S(n)=O(n)



Rappresentazione con puntatori ai figli (nodi **con numero limitato** di figli)

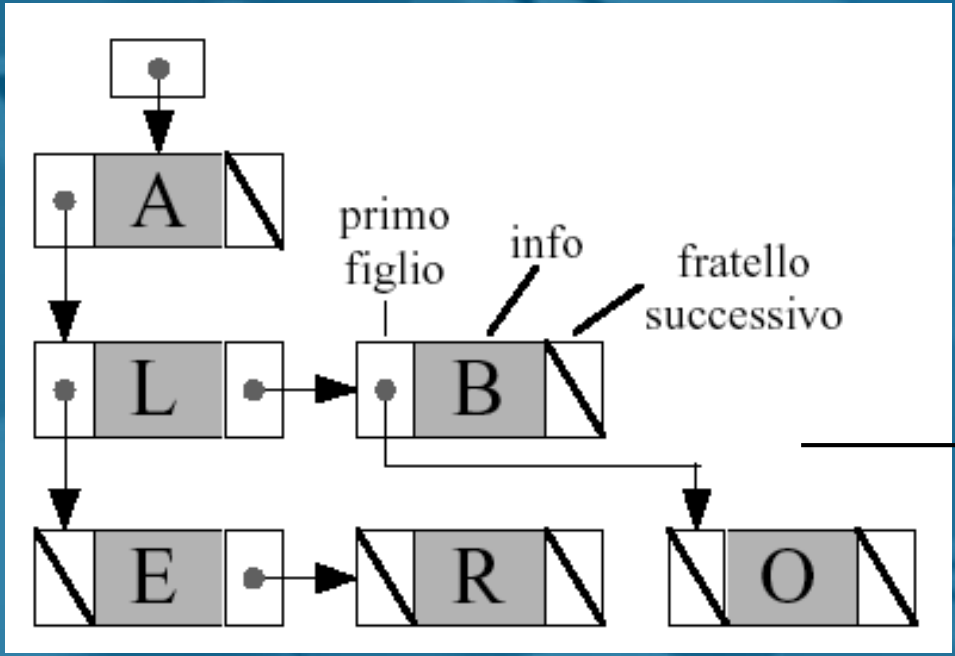
S(n)=O(n)

Rappresentazione con liste di puntatori ai figli (nodi **con numero arbitrario** di figli)



Copyright © 2008 - The McGraw - Hill Companies, srl

Rappresentazioni collegate di alberi



Rappresentazione di tipo **primo figlio-fratello successivo** (nodi con numero arbitrario di figli)

Utile per la rappresentazione binaria di alberi n-ari

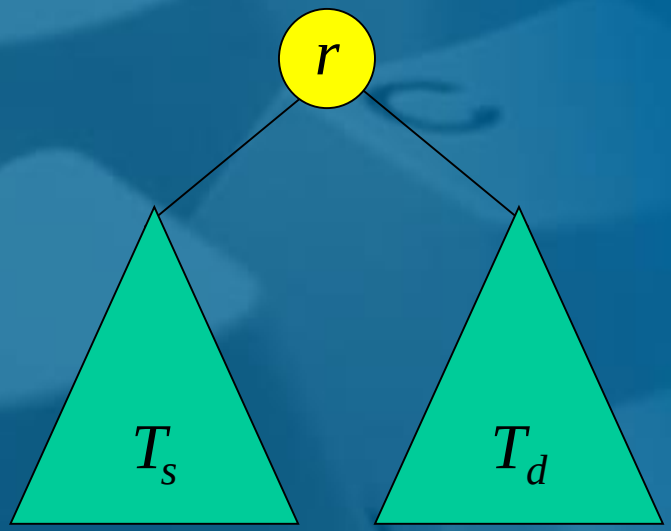
Copyright © 2008 - The McGraw - Hill Companies, srl

Alberi binari

Definizione ricorsiva di albero binario:

a) l'insieme vuoto ϕ è un albero binario □

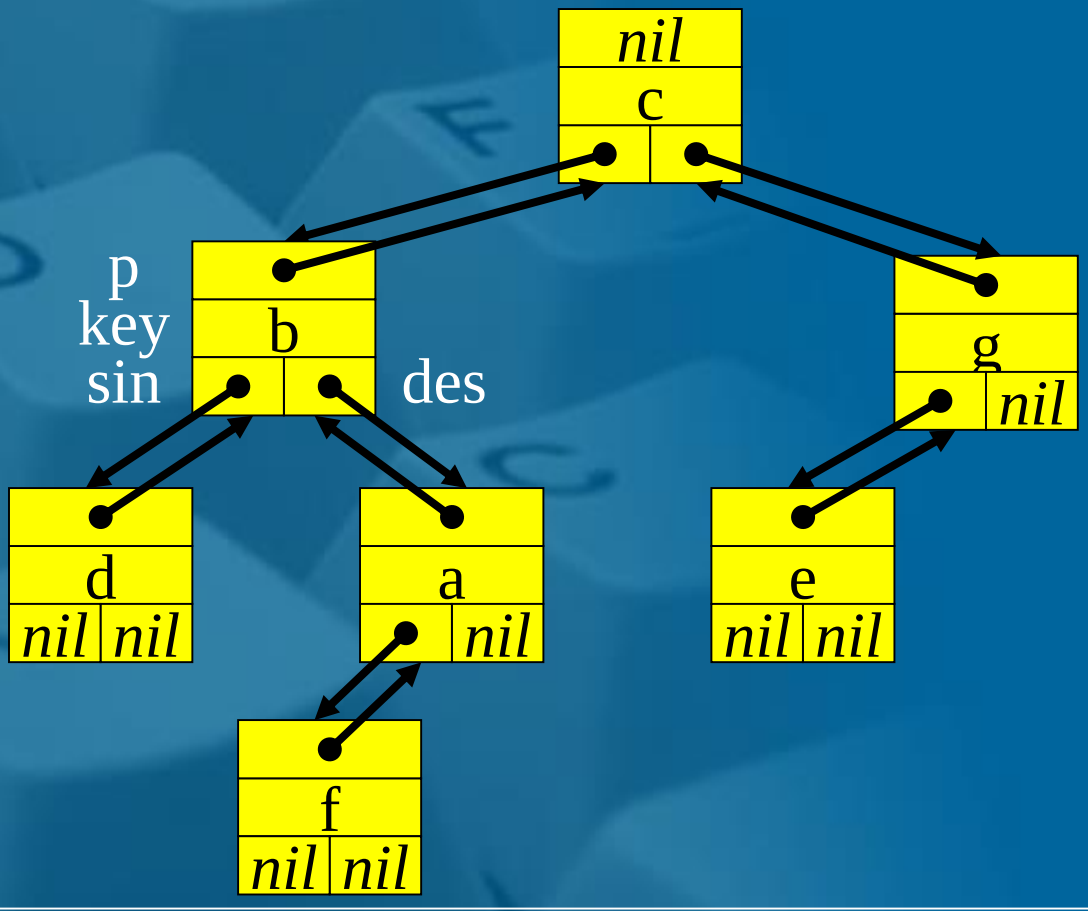
b) se T_s e T_d sono alberi binari ed r è un *nodo* allora la terna ordinata (r, T_s, T_d) è un albero binario



In memoria l'albero binario:

$$T = (c, (b,(d,\phi,\phi),(a,(f,\phi,\phi),\phi)), (g,(e,\phi,\phi),\phi))$$

si rappresenta con due puntatori ai sotto-alberi, ed uno al padre:

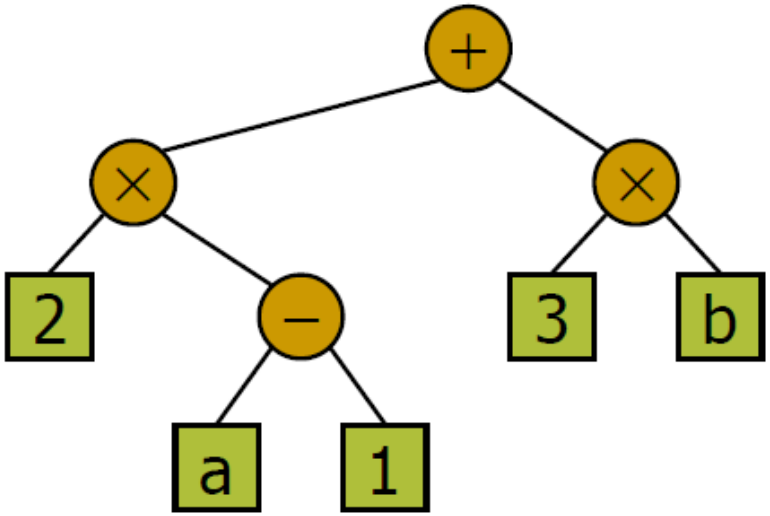


Applicazioni di alberi binari: Espressioni aritmetiche, processi di decisione, ricerca, ecc..

Albero binario associato ad una espressione:

- Nodi interni: operatori
- Nodi esterni: operandi

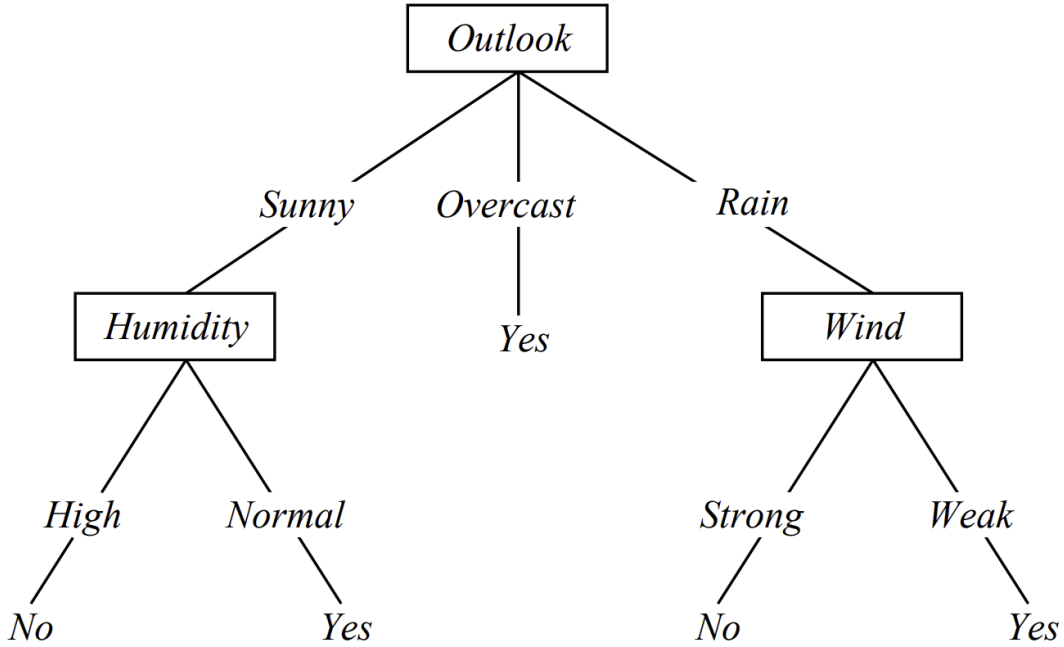
Esempio: $(2 \times (a - 1) + (3 \times b))$



Copyright © 2008 - The McGraw - Hill Companies, srl

Albero di decisione per classificare: ogni nodo interno effettua un test su un attributo, **o**gni foglia assegna una classificazione

Decidere se è la giornata ideale per giocare a Tennis !!



Es. ingresso: (Outlook= *Sunny* , Temperature=*Hot*, Humidity=*High*, Wind=*Strong*).

attributo *valore attributo*

Copyright © 2008 - The McGraw - Hill Companies, srl



Visite di alberi

Algoritmi che consentono l'accesso sistematico ai nodi e agli archi di un albero

Gli algoritmi di visita si distinguono in base al particolare ordine di accesso ai nodi

Copyright © 2008 - The McGraw - Hill Companies, srl



Algoritmo di visita generica

`visitaGenerica` visita il nodo r e tutti i suoi discendenti in un albero

```
algoritmo visitaGenerica(nodo r)
1.    S ← {r}
2.    while (S ≠ ∅) do
3.        estrai un nodo u da S
4.        visita il nodo u
5.        S ← S ∪ { figli di u }
```

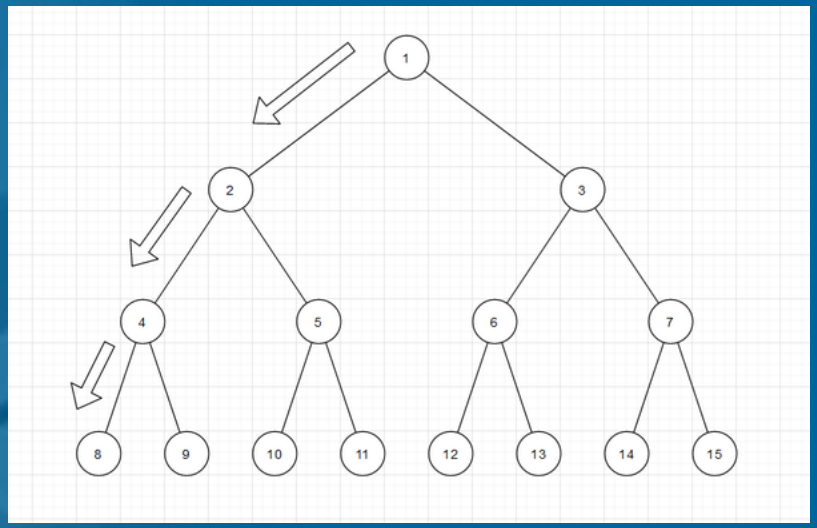
Richiede tempo O(n) per visitare un albero con n nodi a partire dalla radice

Copyright © 2008 - The McGraw - Hill Companies, srl

Algoritmo di visita in profondità

L'algoritmo di visita in profondità (DFS) parte da r e procede visitando nodi di figlio in figlio fino a raggiungere una foglia.

Retrocede poi al primo antenato che ha ancora figli non visitati (se esiste) e ripete il procedimento a partire da uno di quei figli.



Copyright © 2008 - The McGraw - Hill Companies, srl

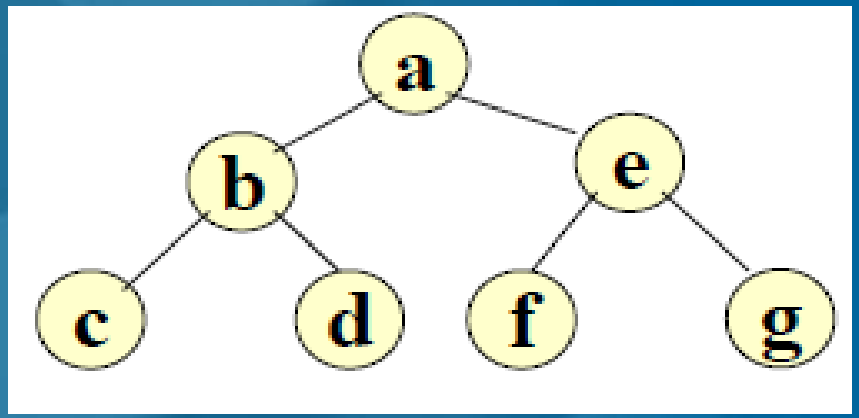


Tre varianti dell'algoritmo di visita in profondità

- Con un albero binario sono possibili 3 strategie:
  - *preordine* o ordine anticipato: si visita prima il nodo e poi i sottoalberi sinistro e destro
  - *inordine* o ordine simmetrico: si visita prima il sottoalbero sinistro e poi il nodo e poi il sottoalbero destro
  - *postordine* o ordine posticipato: si visita prima il sottoalbero sinistro, poi quello destro e poi il nodo

Tre varianti dell'algoritmo di visita in profondità

Esempio:



- Preordine: a b c d e f g
- Inordine: c b d a f e g
- Postordine: c d b f g e a

Copyright © 2008 - The McGraw - Hill Companies, srl



Algoritmo di visita in profondità

Versione iterativa (per alberi binari): visita in preordine

```
algoritmo visitaDFS(nodo r)
    Pila S
    S.push(r)
    while (not S.isEmpty()) do
        u ← S.pop()
        if (u ≠ null) then
            visita il nodo u
            S.push(figlio destro di u)
            S.push(figlio sinistro di u)
```


Copyright © 2008 - The McGraw - Hill Companies, srl



Algoritmo di visita in profondità

Versione ricorsiva (per alberi binari):
visita in preordine

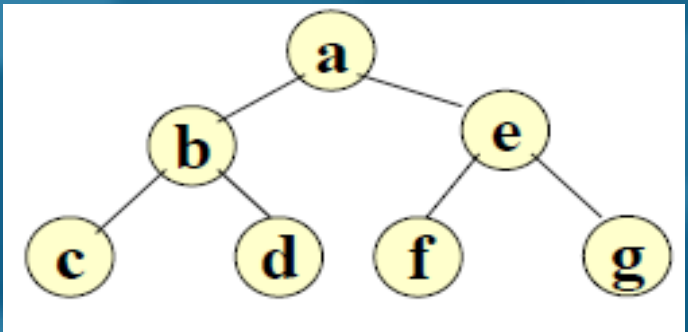
```
algoritmo visitaDFSRicorsiva(nodo r)
1.    if (r = null) then return
2.    visita il nodo r
3.    visitaDFSRicorsiva(figlio sinistro di r)
4.    visitaDFSRicorsiva(figlio destro di r)
```


Copyright © 2008 - The McGraw - Hill Companies, srl

Algoritmo di visita in ampiezza

L'algoritmo di visita in ampiezza (BFS) parte da r e procede visitando nodi per livelli successivi.

Un nodo sul livello i può essere visitato solo se tutti i nodi sul livello i-1 sono stati visitati.



a b e c d f g

Copyright © 2008 - The McGraw - Hill Companies, srl



Algoritmo di visita in ampiezza

Versione iterativa (per alberi binari):

```
algoritmo visitaBFS(nodo r)
    Coda C
    C.enqueue(r)
    while (not C.isEmpty()) do
        u ← C.dequeue()
        if (u ≠ null) then
            visita il nodo u
            C.enqueue(figlio sinistro di u)
            C.enqueue(figlio destro di u)
```


Copyright © 2008 - The McGraw - Hill Companies, srl



Implementazione del tipo Albero Binario

Vedi codice sorgente:

- AlberoBinario.java (interface)
- AlberoBinarioImpl.java (classe albero binario)
- NodoBinario.java (classe nodo)
- AlberoBinarioDemo (classe driver test)

Copyright © 2008 - The McGraw - Hill Companies, srl



Riepilogo

- Nozione di tipo di dato come specifica delle operazioni su una collezione di oggetti
- Rappresentazioni indicizzate e collegate di collezioni di dati: pro e contro
- Organizzazione gerarchica dei dati mediante alberi
- Rappresentazioni collegate classiche di alberi
- Algoritmi di esplorazione sistematica dei nodi di un albero (algoritmi di visita)

Copyright © 2008 - The McGraw - Hill Companies, srl